Remaining Work — Group 20

AMS 560 · Big Data Systems, Algorithms and Networks · Fall 2026

"Prioritize resolving integration issues and completing end-to-end comparisons." — Professor

---

Quick Status — All Members

Member	Role	Code Status	Blocking Item
Jyotir	Filter + Baselines	**Complete (bug fixed)**	Baseline decision logs — needs Riya's grid
Krupesh	Router + Integration	**2 bugs to fix**	jyotir_adapter.py: stale comments + wrong path
Parikshit	Simulator + Traces	**1 crash bug to fix**	node.py raises instead of calling on_outcome
Sur	LinUCB + Reward + Sweep	**Complete**	Team reward-weights decision before final run
Riya	Experiments + Report	**Grid not yet run**	Run 420-grid; write Eval / Results / Discussion

The end-to-end pipeline is built and has been validated on a small mock run. The professor's remark drives a strict priority: (1) fix the two integration bugs that break standalone runs, (2) fix the simulator crash, (3) run the full 420-run experiment grid, (4) generate figures and write the final report.

Jyotir — Filter and Baselines

All deliverables complete. One regression was found by Sur in RESULTS.md and fixed on 2026-11-10.

Bug fixed (2026-11-10) — kv_cache_max_frac not enforced in token-exact path

The token-exact KV memory check in **filter.py** Rule 3 compared the request's token footprint against 100 % of capacity instead of the configured **kv_cache_max_frac** (0.90). A node at 85 % fill would accept a request that pushed usage to 95 %, violating the safety headroom.

What	Detail
File	feasibility_filter/filter.py — Rule 3 (Memory)
Old code	kv_used_tokens + tokens_needed > kv_capacity
Fix	kv_used_tokens + tokens_needed > kv_capacity * kv_limit
New test	TestTokenAwareMemoryCheck :: test_token_check_respects_kv_cache_max_frac
Result	**28 / 28 tests pass**

Remaining open item

Baseline decision logs — running all 6 baselines on all 6 scenarios to produce the CSV / Parquet logs that Riya's analysis needs. No additional code required: the baselines are already registered as **jyotir:round_robin**, **jyotir:min_rtt**, etc. in Krupesh's registry and appear in **config_simulator.json**. The logs will be produced as a side effect of Riya's 420-run grid once the pipeline is verified green.

Coordination items for teammates

Item	Owner	Priority
Fix stale F1/F2 bug comments in jyotir_adapter.py lines 13–16 (both bugs already fixed)	Krupesh	**Medium**
Fix default JYOTIR_DIR resolving to wrong path in jyotir_adapter.py line 29	Krupesh	High
Spot-check S2/S4 (1 seed each) after kv_cache_max_frac fix to confirm sweep numbers stable	Sur	Low
Team decision: reward weights (energy scale) before final runs	Team	**High**

Krupesh — Router Service and Integration Layer

Router service, decision log, policy interface, and integration adapter are complete. Two bugs in **jyotir_adapter.py** must be fixed before the router can run standalone without Riya's path override.

Bug A — Stale F1/F2 comments (documentation)

File: **integration/jyotir_adapter.py**, lines 13–16

The comment block documents F1 (warming node skips safety rules) and F2 (memory ignores request size) as "known behaviour". Both were fixed by Jyotir (commit 59b63af). A grader reading the adapter will think the bugs still exist.

Fix: Remove lines 13–16 and replace with a one-line note, e.g.:

```
# Jyotir's filter (commit 59b63af+): warming nodes run all 5 rules;

# memory rule enforces kv_cache_max_frac, not 100 % capacity.
```

Bug B — Default JYOTIR_DIR resolves to wrong path (runtime)

File: **integration/jyotir_adapter.py**, line 29

```
JYOTIR_DIR = Path(__file__).resolve().parents[2] / "Jyotir"
```

Because Krupesh's folder is uploaded one level deeper than the repo root, **parents[2]** resolves to **Krupesh/geo-inference-scheduler/** and **JYOTIR_DIR** becomes **Krupesh/geo-inference-scheduler/Jyotir/**, which does not exist. Running **uvicorn router_service.app:app** standalone fails at import time. It works only because Riya's runner overrides this variable before importing.

Fix: Walk up to the repo root by detecting a known sentinel (e.g. the **Parikshit/** folder), or use **parents[4] / "Jyotir"** to match the actual depth.

Optional — telemetry-age accounting (team decision)

Sur's sweep found ~3,200 true-state power-limit violations per S2 run with the per-node W = 500 policy, all caused by the router's 1-second-old telemetry. The router already counts in-flight placements against load; applying the same accounting to power draw and temperature would reduce this. Requires Parikshit's per-request power delta. Team decision before final runs.

Parikshit — Simulator and Event Traces

Simulator, telemetry schema v1, all 6 scenario traces, 122 tests, and the KrupeshRouter integration bridge are complete. One crash bug must be fixed before the final grid.

Bug — node.py crashes on KV overflow instead of recording a graceful failure

File: **simulator/node.py**, lines 178–179

```
if job.kv_tokens > self.kv_capacity:

raise ValueError(f"{job.request['request_id']} needs more KV cache ...")
```

If a request slips past the filter with more tokens than the node's total KV capacity (possible with stale telemetry during a burst), the simulator raises an uncaught **ValueError** and crashes the entire run, losing all results collected so far.

Fix: Call the failure callback — the same pattern as the go_down() path:

```
if job.kv_tokens > self.kv_capacity:

self.on_outcome(self._failure(job, self.env.now))

return
```

Priority: High — this would crash Riya's 420-run grid if any run hits this edge case.

Optional / team decisions

Item	Who decides	Impact if changed
S2 burst hardness: raise base rate so the 3× burst exceeds cluster max throughput	Team	Sur sweep no longer directly comparable; re-run needed
Telemetry tick: reduce 1 s → 0.5 s to halve true-state power violations	Team + Krupesh	Minor performance change; must coordinate with router

Sur — LinUCB Policy, Reward, and Window Sweep

All code, the 1,350-run window sweep on the real pipeline, and the full set of sweep tables and figures are complete. One team decision blocks finalising the S3 results.

Team decision required — energy weight for S3 price-aware routing

Current reward weights: SLO = 1.0, latency = 0.2, energy = 0.1, thermal = 0.1. With these values the learned policy gives **identical results in S3 (price shift) and S1 (stable)**. The energy term moves the reward by only ~0.01 per request, so the policy never shifts traffic to cheaper sites when California's prices jump to $0.046–0.050 / kWh.

Option	What to change	Effect on S3	Effect on S1 goodput
Keep current weights	Nothing	No price-aware routing; only lowest-energy reacts	72.2 req/s — unchanged
Raise energy weight	Increase energy YAML weight or energy_cost_max_usd	Policy shifts to cheaper nodes after price jump	May lose ~1–2 req/s — SLO no longer fully dominates

> **Config:** Sur/configs/policies/sw_linucb.yaml. **Must be decided before Riya's final grid run** — changing after the run means re-running all 420 jobs.

Optional — spot-check after Jyotir's kv_cache_max_frac fix

The window sweep ran with Jyotir's pre-fix filter. The fix tightens the memory check by 10 % of capacity (nodes between 90–100 % KV fill that were accepted are now rejected). Sur's RESULTS.md notes no S6 true-state violations involved the warming node. A 1-seed spot check on S2 and S4 is recommended to confirm numbers are stable before the full grid.

Key numbers from the sweep (for the report)

Finding	Number
W=500 per-node vs best baseline (lowest-energy) — S1 goodput	72.2 vs 61.1 req/s
Adaptation time S2 — W=500 per-node vs plain LinUCB	20.4 s vs 33.6 s [95 % CI]
Hard-rule violations by filter-allowed nodes (all 1,350 runs)	**0**
True-state power violations per S2 run — W=500 (telemetry age, not policy)	~3,200
Cost per SLO-met request S3 — W=500 vs round-robin	268 µ$ vs 677 µ$

Riya — Experiments, Metrics, and Final Report

The end-to-end experiment pipeline is complete: simulator → router → filter → policy → decision log → metrics → recovery → bootstrap CIs → figures. The 420-run grid has not yet been executed. The Evaluation, Results, and Discussion sections of the final report are not yet written.

Step 1 — Pipeline validation (run first, ~5–10 min)

```
python run_evaluation.py \

--config config_validation.json \

--output ../results/validation_01
```

1 policy × 3 scenarios × 1 seed = 3 runs. After it finishes check: all three **status.json** files show status = ok; **decision_log.csv** passes validate_log(); **true_state_violation** column is present. **Requires Parikshit's node.py crash fix and Krupesh's JYOTIR_DIR fix first.**

Step 2 — Full 420-run grid (~21 min on 16 cores)

```
python run_evaluation.py \

--config config_simulator.json \

--output ../results/final_01
```

7 policies × 6 scenarios × 10 seeds = 420 runs. Use **--resume** if interrupted (each completed bundle is SHA-256 verified before skip). After run: check **run_coverage.json** shows complete = true, and **pairing_checks.csv** shows workloads_match = true for all seeds. **Requires team reward-weights decision first.**

Step 3 — Generate figures

```
python figures/plot_results.py --results ../results/final_01
```

Produces goodput-over-time curves and 95 % bootstrap CI bar charts for 9 key metrics. Requires all 10 seeds to have ci_status = available.

Step 4 — Report sections to write

Section	Inputs needed	Status
Evaluation setup	config_simulator.json (frozen)	**Not started**
Results — goodput & adapt.	final_01/figures/ and recovery_summary.csv	**Not started**
Results — cost & energy	confidence_intervals.csv	**Not started**
Ablation runs (3 configs)	no-filter / no-context / no-forgetting run configs	**Not started**
Discussion	After results sections	**Not started**
Final report assembly	All 5 member sections + evaluation sections above	**Not started**

Three reporting caveats to include in the paper:

(1) Lost goodput is null until a matched oracle (round-robin baseline curve at same seed) is passed to recovery_metrics(); add this as a post-processing step after the grid.

(2) Report every adaptation time against the S1 stable-noise floor — Sur found 4–12 s pseudo-delays in 4 / 10 seeds on a stable cluster.

(3) S6 needs two anchor times: t0 = 200 s (outage) and t0 = 400 s (node return). Show the returned node's traffic share alongside the adaptation time.

Critical Path to Final Results

These items must be completed in order. Everything else is parallel or optional.

#	Action	Owner	Blocks
1	Fix node.py crash — replace raise with on_outcome callback	Parikshit	Riya's grid
2	Fix jyotir_adapter.py JYOTIR_DIR default path	Krupesh	Standalone router; Riya's pipeline
3	Team reward-weights decision for S3 (energy scale)	Team	Meaningful S3 results
4	Run config_validation.json — 3-run smoke test	Riya	Full grid confidence
5	Run full 420-run grid with config_simulator.json	Riya	Figures, results, paper
6	Generate figures (plot_results.py)	Riya	Paper figures
7	Write Evaluation + Results + Discussion + Conclusion	Riya	Final submission
8	Assemble final report (all 5 member sections)	Riya	Submission

Parallel tasks (do not block the critical path):

Action	Owner	Priority
Fix stale F1/F2 comments in jyotir_adapter.py lines 13–16	Krupesh	**Medium**
Sur spot-check S2/S4 (1 seed) after Jyotir's kv_cache_max_frac fix	Sur	Low
Team discussion on S2 burst hardness and telemetry tick interval	Team	Low
Provide round-robin oracle curve to unlock lost-goodput metric	Riya	Low